

Product Requirements Document

SyncSphere: Collaborative Event Planning Hub

1. Project Overview

SyncSphere is a collaborative event planning platform that brings scheduling, task assignment, budget tracking, and guest management into a single workspace. This engagement covers full application development in Flutter: a working, installable app built on a scalable architecture, not a design mockup or static screen set. The interface should read as premium and considered card-based dashboards, clear information hierarchy, restrained accent color implemented as real, interactive Flutter widgets backed by real app state.

Product direction, target users (individuals and teams planning personal, community, and corporate events), and general feature scope are already defined. This document defines what the developer is expected to build and ship.

2. Objectives

- Set up a complete, scalable Flutter project with clean architecture, reusable widgets, and a maintainable folder structure.
- Implement every core module listed in Section 4 as a working, navigable screen backed by real local data — not exported images.
- Seed the app with the supplied sample data (events, guests, tasks, budgets, milestones) so screens read as populated on first run.
- Wire a single consistent state-management approach and local persistence so created/edited data survives app restarts.
- Deliver clean, documented source code with setup instructions and basic tests.

3. Assets Provided

The following files are supplied to support the build. Full usage rules appear in instruction.md.

- assets/reference/moodboard_premium_apps.png overall mood and tone reference (inspiration only, not shipped in the app)
- assets/reference/color_palette_reference.png starting color palette reference
- assets/reference/dashboard_layout_reference.png structural layout reference for the dashboard
- assets/reference/icon_style_reference.png icon stroke weight (2px) and corner-radius (20px) reference
- assets/data/sample_event_data.csv sample events, guests, tasks, budget lines, and milestones to seed local storage
- assets/data/brand_tokens.json starting color, typography, spacing, and shape values for the design-token file
- assets/data/content_copy.docx approved onboarding, empty-state, notification, and validation copy

The four reference/ images are inspiration only and are never bundled as visual assets in the shipped app. The three data/ files are working content sources whose values must be reachable at runtime as seeded records, theme tokens, or on-screen copy.

4. Scope of Work — Major Deliverables

#	Major Deliverable	Core Screens / Modules
---	-------------------	------------------------

1	Project Setup & Design Token System	Flutter project scaffold, theme/design_tokens.dart, reusable buttons/inputs/cards, custom icon set
2	Onboarding & Authentication	3 onboarding screens, sign up, log in + social sign-in, forgot password, persistent sessions
3	Event Dashboard & Event Creation/Editing	Dashboard home, create event, edit event, event detail overview
4	Guest & Task Management	Guest list, add/edit guest, task list/board, add/edit task
5	Budget, Timeline & Calendar	Budget summary, budget analytics chart, event timeline, calendar view
6	Notifications, Profile, Settings	Notification center, profile screen, settings screen
7	State, Persistence & Delivery	App-wide state management, local data persistence, tests, README

5. Deliverable Requirements

- The app must run on standard mobile phone frame sizes with responsive layout (no fixed single-size assumption).
- Every screen must consume the color, type, and spacing tokens defined in design_tokens.dart no ad hoc values.
- Screens must be populated with the supplied sample data via a seeding service, not lorem ipsum or placeholder labels.
- Every interactive element (buttons, inputs, cards, list rows) must implement at least a default and a pressed/selected state.
- Empty-state and error/validation messages must use the exact wording supplied in content_copy.docx.
- The full primary flow must be navigable end to end inside the running app: onboarding - sign-in - dashboard - at least one detail screen per module (event, guest, task, budget).
- All data created or edited by the user (events, guests, tasks, budget entries) must persist across app restarts.

6. Naming & Project Structure

Follow the exact folder and file structure defined in instruction.md Section 5 (lib/, assets/, test/, pubspec.yaml, README.md). Dart files use snake_case; classes use UpperCamelCase.

7. Deliverables Checklist

- Flutter project scaffold with clean architecture and design-token theme file
- Onboarding (3 screens) and authentication screens (sign up, log in, social sign-in, forgot password) with real validation
- Dashboard home and event creation/editing flow, backed by local storage
- Guest management screens with working RSVP states and search/filter
- Task management screens with priority, assignee, and progress states
- Budget summary and analytics chart, timeline, and calendar screens
- Notification center, profile, and settings screens with working toggles and log-out
- Single app-wide state-management approach and local persistence layer
- README.md with setup/run instructions and basic widget/unit tests